NLP 课程小组作业 · 第一次个人汇报 · 题目九

From Code Assistants to Code Guardians: Real-Time Hallucination Detection for AI-Generated Code in CI/CD Pipelines

相关文献综述：代码幻觉的分类、检测与 CI/CD 集成

小组成员：李甘、张星烁、连瑾昭
汇报人：李甘

题目来源： openi.pcl.ac.cn/luckh2/nlp26projects
仓库： github.com/Nictheboy/college-nlp-lab

起点：一次由 AI Agent 引发的生产事故

Amazon 2026 年 AI Agent 故障事件

▸ 2026 年 3 月， Amazon 电商站点连续出现严重宕机

– 3 月 5 日：订单量骤降约 99%，估计损失 630 万笔订单

– 3 月 2 日：另一起事件导致 120 万次报错、 12 万笔订单丢失

▸ 根因指向工程师采纳了 AI Agent 基于过时内部文档给出的“自信但错误”的建议

▸ 同期 AWS 内部也曾因 Kiro 智能编码工具的自主改动引发长达 13 小时的服务中断

▸ Amazon 随后启动 335 个核心系统的 90 天代码安全整改，要求高级工程师对 AI 辅助代码进行强制复核

问题的本质

▸ 现有研究多聚焦自然语言的“事实性幻觉”

▸ “代码幻觉”——生成代码看似正确，实则存在逻辑漏洞 / 依赖过时或不存在的 API——检测研究仍属空白

▸ 传统 CI/CD 依赖人工 Code Review + 规则化 Lint，难以覆盖 LLM 特有的错误模式

▸ 核心矛盾：检测覆盖率 vs. 流水线执行速度

题目九核心要求与三大创新点

核心任务

开发一个轻量级代码验证器，集成到 CI/CD 流水线中：在 AI 生成代码被合并前，
结合静态分析 + 动态沙箱测试，检测潜在的运行时错误与 API 兼容性问题，
并在检测覆盖率与执行效率之间取得平衡。

创新点一

代码幻觉分类体系

首次系统性定义 LLM 生成代码中的幻觉类型： API 误用、逻辑短路、资源泄漏、安全漏洞等

创新点二

静态 - 动态混合验证

AST 模式匹配（快） + 轻量级沙箱执行（准）相结合，实现高效的代码审查

创新点三

CI/CD 原生集成

以结构化反馈形式将验证结果返回给开发者，支持一键修正建议

文献地图： 2025 年及以后、聚焦软件工程实践的 13 篇文献

① 静态 / 动态验证与沙箱安全

- ▸ Static Analysis for Library Hallucinations (2026)
- ▸ Static-Pass Dynamic-Fail Gap (2026)
- ▸ SandboxEval (2025)
- ▸ Package Hallucinations / Slopsquatting (USENIX Sec'25)

② 仓库级文档 / 测试自动维护

- ▸ DocAgent (2025)
- ▸ CASCADE (2026)
- ▸ Repository Intelligence Graph (2026)
- ▸ ReCite · Linux 内核过时注释修复 (ASE'26)

③ AI 时代软件工程 与 大仓库长时程 benchmark

- ▸ SWE-Milestone (2026)
- ▸ SWE-EVO (2025)
- ▸ RoadmapBench (2026)
- ▸ Agentic SE Roadmap (2025)
- ▸ Agentic Code Review (2026)
- ▸ LLM Agents for Dependency Upgrades (2025)

三条主线分别回答：幻觉“怎么被发现” → “文档 / 测试怎么被自动同步” → “这类 agent 放到大仓库长时程场景下够不够用”

大仓库 · 长时程

三个 2025-26 年 benchmark 共同证明： agent 在长时程演化场景下全线失灵

SWE-Milestone (2603.13428) · SWE-EVO (2512.18470) · RoadmapBench (2605.15846)

▸ SWE-Milestone： 把真实提交历史构造成“里程碑 DAG”，测连续任务序列——分数从孤立任务的 >80% 骤降到连续场景的 38.03%

▸ SWE-EVO： 源自 7 个成熟开源项目的 release notes，任务平均涉及 21 个文件、 874 个测试—— GPT-5.4+OpenHands 仅 25% （同模型在 SWE-Bench Verified 上是 72.80% ）

▸ RoadmapBench： 17 个仓库、 115 个跨版本升级任务，最强模型 Claude-Opus-4.7 也只解决 39.1%

启示 现有 agent 在“单次、孤立”任务上尚可，一旦放到连续演化的真实仓库里就大幅失灵——这正是本项目要瞄准的场景。

真实案例 · 大仓库

ReCite： Linux 内核里 869 处“没人发现”的过时注释

Sun, Lyu, Ma, Kuang, Widyasari, Zhang, Ma, Lawall, Lo · arXiv:2608.03734, ASE 2026

▸ 三阶段方法：检测注释中引用的函数符号是否已在代码中失效
→ 追溯 Git 历史定位该符号的演变 → 用 LLM 结合历史上下文生成修复建议

▸ 在 Linux 内核 v6.18-rc1 上检测出 869 处过时的函数引用注释

▸ 人工评估： 89.0% 的修复建议有用， 42.5% 可直接采用；

提交 75 个补丁，其中 50 个已被上游接受

启示 证明“文档 / 注释漂移”在顶级长期维护的大仓库中真实存在且规模可观，而现有做法仍主要靠事后人工审计。

静态检测 · 局限

静态分析的能力边界：库幻觉检测实证研究

Miranda-Pena et al. · arXiv:2604.07755 (2026)

- LLM 生成代码中引用不存在的库 / 接口的比例高达 8.1%–40%
- 纯静态分析工具能检出 16%–70% 的错误、 14%–85% 的库幻觉，

  检出率随模型和数据集大幅波动
- 人工分析给出理论上限：静态方法最多能解决 48.5%–77% 的问题
- 结论：静态分析是“性价比高的部分方案”，但无法独立解决问题

启示 直接支撑题目“为什么需要混合验证”——静态分析快但覆盖率有硬上限。

动态检测 · 沙箱

静态通过 ≠ 运行时安全： Static-Pass Dynamic-Fail 现象

Pourleyli, Das Urmi, Melo · arXiv:2609.10762 (2026)

▸ 三阶段流水线：静态扫描（ Bandit + Semgrep ）→ LLM 驱动的 CWE 推理
→ Docker 隔离沙箱中的自动化漏洞验证

▸ 1,355 个 Python 样本中， 654 个通过了静态扫描

▸ 在这些“静态通过”的样本中，流水线仍找出 394 个候选漏洞，

其中 95 个被确认可被实际利用

▸ 结论：静态通过与运行时安全是两个层级的保证，不可互相替代

启示 为题目的“动态沙箱测试”环节提供了可复用的三阶段流水线架构范式。

沙箱与供应链

沙箱执行安全 与 依赖投毒风险

SandboxEval (arXiv:2504.00018) · Package Hallucinations (USENIX Sec 2025)

▸ SandboxEval：针对“执行不可信 LLM 生成代码”场景设计安全测试集，
覆盖敏感信息泄露、文件系统篡改、非法外部通信等风险点

▸ Package Hallucinations：测试 16 个代码模型，包幻觉平均发生率 19.6%

（开源模型约 21%，商业模型约 5%），发现 20.5 万 + 个虚构包名

▸ 约 45% 的虚构包名在重复查询中持续出现——

攻击者可预先抢注这些包名实施“ slopsquatting” 供应链攻击

启示 提示 CI/CD 集成方案必须同时保证“沙箱本身的安全边界”与“依赖来源可信”。

把切口收窄：文档 / 测试“漂移”是幻觉的一个被忽视的诱因

回到起点案例

Amazon 3 月 5 日事故的根因，官方复盘为：工程师采纳了 AI Agent 基于“过时内部 wiki” 推断出的建议——本质上就是一次“文档已经漂移、但没人（也没有工具）提醒它已经过时”的失败。

我们观察到的缺口

▸ 静态 / 动态验证文献（第一组）解决的是“这段新代码本身是否正确”，而不是“这段代码改动是否让别处的说明或用例变得过时”

▸ 文档 / 测试维护文献（第二组）已经能生成或修复文档，但止步于函数 / 文件级别，

没有仓库级架构图，也没有跨模块的传播判断机制（详见下一页对比）

▸ 长时程 benchmark （第三组）证明： agent 一旦离开孤立任务、进入真实的

持续演化场景，表现会大幅下降——而“改代码后维护文档 / 测试”正是这种

长时程、跨模块场景的典型代表，却几乎没有工作把它当作 benchmark 目标

已有的文档 / 测试自动维护工作 —— 以及它们止步的地方

ReCite

arXiv:2608.03734

做了什么

在 Linux 内核中追溯 Git 历史，检测并修复过时的函数引用注释

止步于：只处理“注释里的失效符号引用”，无架构图，不更新测试

DocAgent

arXiv:2504.08725

做了什么

Reader/Searcher/Writer/Verifier
多智能体拓扑式生成文档

止步于：从零生成文档，不是“ diff→ 判断是否级联更新”

CASCADE

arXiv:2604.19400

做了什么

用文档生成单测、用代码生成参照
实现，双向交叉验证一致性

止步于：单文件 / 单函数级一致性
检测，无仓库级传播机制

Repo Intelligence Graph

arXiv:2601.10112

做了什么

从构建系统抽取确定性架构图，喂给编码 agent 提升单次任务表现

止步于：架构图只服务单次编码任务，不驱动文档 / 测试维护流程

架构感知的模块级文档 / 测试同步 Agent

Step 0 · 离线

抽取仓库架构

解析目录结构 /import 关系，聚类出组件，用 LLM 为每个组件生成一份“组件说明文档”
（职责、接口、依赖关系）

Step 1

diff → 模块归属

给定一次 PR/commit 的 diff，
依据架构图把改动文件
映射到其所属的一个或多个组件

Step 2

模块级 agent 更新

为每个受影响组件启动一个 agent，读取该组件已有说明文档 + diff，判断 / 更新文档，并更新或新增对应测试

Step 3

级联传播决策

agent 依据架构图中的依赖边，
查看相邻组件的说明文档，
自主判断这些组件是否也
需要同样处理（设深度上限）

关键设计取舍

Step 3 的“是否传播”由 agent 结合两端组件文档自主判断，而非单纯按图遍历全部邻居——避免更新无限扩散，同时保留处理跨模块真实影响的能力。

创新点：据我们检索，尚未有工作同时做到这四件事

维度	DocAgent / ReCite	CASCADE	Repo Intelligence Graph	我们的方法
分析粒度	函数 / 对象级	函数 / 文件级	可构建组件级（服务单次编码）	**模块 / 组件级架构图（服务维护流程）**
触发方式	commit 时局部符号引用扫描	对已有代码库批量扫描	构建系统解析，单次任务加载	**diff → 架构图 → 精确定位受影响组件**
产出	仅文档	仅一致性报告	仅架构图	**文档 + 测试同步更新**
跨模块传播	无（仅双向引用）	无	无	**agent 依据架构图自主判断是否级联**

简单研究规划：快速迭代——先能跑、再优化、最后定量 benchmark

第 1 周

跑通最小可用版本（MVP）

选 1 个小型真实仓库；架构划分先用
简单规则（按目录前缀聚类），diff→ 模块归属用路径匹配；单模块
agent 读文档 +diff 生成一版更新（文档 + 至少 1 条新测试），先不做
级联传播。目标：端到端跑通一次

第 2-4 周

逐项优化

架构抽取从规则升级为聚类 +LLM 摘要；引入 CASCADE 式双向校验
提升更新质量；加入级联传播判断（架构依赖图 + agent 自主决策，设深度上限），在更多仓库上试跑并修正明显错误

第 5-6 周

定量 benchmark 评测

参考 SWE-Milestone / SWE-EVO 的
构造方式，从真实仓库的历史 PR 中
挖掘“多模块同时改文档”的连续场景作评测集；定义指标：模块归属准确率、文档 / 测试更新质量、级联判断准确率 / 召回率、延迟与 token 成本

第 7-8 周（如时间允许）

CI 集成与打磨

封装为 GitHub Action；在评测集上
跑完整流程、对比人工真实更新；整理结果与局限性，撰写最终报告

核心原则：不追求一步到位的完美架构抽取或传播算法，先用最简单的规则拉通全流程，用真实运行结果指导下一步优化。

参考文献列表

- [1] Miranda-Pena et al. An Empirical Analysis of Static Analysis Methods for Detection and Mitigation of Code Library Hallucinations. arXiv:2604.07755 (2026)
- [2] Pourleyli, Das Urmi, Melo. Beyond Static Guarantees: Measuring the Static-Pass Dynamic-Fail Gap in Security-Sensitive and LLM-Generated Python Code. arXiv:2609.10762 (2026)
- [3] Rabin et al. SandboxEval: Towards Securing Test Environment for Untrusted Code. arXiv:2504.00018 (2025)
- [4] Spracklen et al. We Have a Package for You! Package Hallucinations by Code Generating LLMs. USENIX Security 2025
- [5] DocAgent: A Multi-Agent System for Automated Code Documentation Generation. arXiv:2504.08725 (2025)
- [6] Kiecker et al. CASCADE: Detecting Inconsistencies between Code and Documentation with Automatic Test Generation. arXiv:2604.19400 (2026)
- [7] Cherny-Shahar, Yehudai. Repository Intelligence Graph: Deterministic Architectural Map for LLM Code Assistants. arXiv:2601.10112 (2026)
- [8] Sun et al. We Must Have Missed This Comment: Detecting and Repairing Stale Function References in Linux Kernel Comments (ReCite). arXiv:2608.03734, ASE 2026
- [9] Deng et al. SWE-Milestone: Evaluating AI Agents on Continuous Software Evolution. arXiv:2603.13428 (2026)
- [10] Le et al. SWE-EVO: Benchmarking Coding Agents in Long-Horizon Software Evolution Scenarios. arXiv:2512.18470 (2025)
- [11] Xu et al. RoadmapBench: Evaluating Long-Horizon Agentic Software Development Across Version Upgrades. arXiv:2605.15846 (2026)
- [12] Hassan et al. Agentic Software Engineering: Foundational Pillars and a Research Roadmap. arXiv:2509.06216 (2025)
- [13] Kamalı et al. Rethinking Code Review in the Age of AI: A Vision for Agentic Code Review. arXiv:2605.17548 (2026)
- [14] Tawosi, Alamir, Liu, Veloso. LLM Agents for Automated Dependency Upgrades. arXiv:2510.03480 (2025)